

OMNISEARCH ENGINE

OmniSearch Engine

Распределенная On-Premise система семантического поиска по видеоматериалам на базе технологии RAG

Команда 39А: Денис (TL / Arch / Go), Паша (Backend / Kotlin), Миша (ML / Python), Кирилл (Frontend / React), Максим (DevOps / Infra)

Проблема И Концепция Решения

⚠ Проблема видеоархивов

- ♦ **Взрывной рост данных:** Корпоративные видеоархивы увеличиваются в объеме без возможности структурирования.
- ♦ **Неэффективная разметка:** Ручное тегирование контента стоит дорого и подвержено человеческому фактору.
- ♦ **Слепая зона метаданных:** Поиск ограничен исключительно названиями файлов или базовым описанием.

✅ Решение OmniSearch

- ♦ **Асинхронная сборка:** Разделение видеоряда на аудиодорожку и ключевые кадры для параллельной обработки.
- ♦ **Семантическая транскрибация:** Автоматический перевод речи в текстовые тайм-кодированные чанки.
- ♦ **Многомерный поиск:** Поиск релевантных сегментов на естественном языке с точностью до секунды.

Архитектура И Пайплайны

⚙️ Ingestion Pipeline

- ◆ **Загрузка в MinIO:** Исходный видеофайл поступает в S3-хранилище как единый источник правды (SSOT).
- ◆ **Декомпозиция медиапотока:** Параллельное выделение кадров (GoCV) и WAV-аудио (FFmpeg).
- ◆ **ML-транскрибация:** Обработка через Faster-Whisper, генерация векторов E5-base и индексация в Qdrant.

🔍 Retrieval Pipeline

- ◆ **Векторизация запроса:** Текстовый поисковый запрос пользователя транслируется в 768-мерный вектор.
- ◆ **HNSW-поиск:** Qdrant осуществляет высокоскоростное сопоставление по векторному пространству.
- ◆ **Обогащение метаданных:** Ktor подтягивает пути кадров из PostgreSQL и возвращает готовые карточки с превью.

Медиа-Движок: Video Engine

Системный декодинг на Go

- ◆ **GoCV CGO биндинги:** Прямой вызов C++ библиотеки OpenCV 4 без дополнительных накладных расходов.
- ◆ **Контроль памяти:** Явное ручное освобождение ресурсов через `defer close()` для объектов `Mat`.
- ◆ **Мгновенное извлечение:** Сэмплирование кадров со скоростью выше 100 кадров в секунду.

Экстракция аудио и оптимизации

- ◆ **Копирование без транскода:** Использование встроенного подпроцесса FFmpeg для быстрого экспорта WAV.
- ◆ **Минимальный оверхед:** Видео-движок работает как HTTP-сервис со строгим лимитом в 500 MB RAM.
- ◆ **Обложки на лету:** Первый успешный кадр автоматически преобразуется в превью-обложку.

Оркестрация: Backend На Ktor

Kotlin Control Plane

- ◆ **Стек технологий:** Высокопроизводительный асинхронный сервер Ktor на JVM 21, DI через Koin.
- ◆ **База метаданных:** Использование PostgreSQL 15, взаимодействие через легковесный фреймворк Exposed.
- ◆ **Безопасность потоков:** Генерация Presigned URLs в MinIO для защищенного стриминга медиа-частей.

Конечный автомат обработки

- ◆ **Логирование состояний:** Контроль статуса загрузок от постановки в очередь до успешного завершения.
- ◆ **Событийный подход:** Регистрация вебхуков-колбэков от медиа-движка и нейросетевого ядра.
- ◆ **Проксирование поиска:** Ktor принимает поисковые запросы, делегирует в ML и дополняет результаты.

Интеллектуальное Ядро: ML

Speech-to-Text & Chunking

- ◆ **Faster-Whisper large-v3:** Сверхточная локальная транскрибация аудиоряда в текстовый формат.
- ◆ **Метод скользящего окна:** Текст нарезается на перекрывающиеся логические сегменты (чанки).
- ◆ **Синхронные таймкоды:** Сохранение точных секунд начала и окончания для каждого выделенного чанка.

Векторный поиск в Qdrant

- ◆ **Модель E5-base:** Перевод текстовых чанков в плотные 768-мерные векторы-эмбединги.
- ◆ **Векторная СУБД Qdrant:** Индексация данных с полезной нагрузкой (payload: video_id, timestamps).
- ◆ **Субсекундный поиск:** Поиск по HNSW-индексам с задержкой p95 менее 500 миллисекунд.

Интерфейс: React SPA

Интерфейс на React 19

- ◆ **Современная экосистема:** Быстрый сборщик Vite, строгая типизация TypeScript, роутер React Router DOM.
- ◆ **Удобный UX при загрузке:** Drag-and-Drop зона для видео с плавным интерактивным прогресс-баром.
- ◆ **Zustand & React Query:** Управление стейтом и фоновое отслеживание (polling) статуса готовности видео.

Интерактивный медиаплеер

- ◆ **Прямая адресация:** Клик по найденной текстовой цитате мгновенно запускает видео с нужного фрагмента.
- ◆ **Безопасное вещание:** Интеграция с HTML5-плеером на основе временных S3 Presigned URLs.
- ◆ **Плавный интерфейс:** Скелетоны и заглушки исключают визуальные задержки при обновлении контента.

Инфраструктура И CI/CD

Docker Compose Среда

- ◆ **Полная изоляция:** 8 сервисов, запущенных в единой виртуальной сети omnisearch-network.
- ◆ **Upstream Nginx Proxy:** Точка входа для маршрутизации запросов Frontend, API и S3-хранилища.
- ◆ **GPU Passthrough:** Проброс физической GPU Nvidia в Docker для ускорения инференса ML-моделей.

>_ Автоматизация CI/CD

- ◆ **Проверка качества:** Линтинг и мультитек-сборка в GHA (Gradle, Node.js, Go, Python).
- ◆ **Docker Buildx Bake:** Оптимальная сборка образов с переиспользованием GHA Cache.
- ◆ **Self-hosted Runner:** Быстрый автоматический деплой на удаленный выделенный сервер.

Результаты MVP И Планы

Текущие итоги MVP



Дорожная карта

- ◆ **Замкнутый пайплайн:** Автоматический разбор, индексация и поиск видео.
- ◆ **Субсекундный поиск:** Отработка сложного векторного сопоставления за долю секунды.
- ◆ **Мультимодальность:** Внедрение визуальных моделей CLIP для распознавания объектов.
- ◆ **Whisper CTranslate2:** Оптимизация инференса для снижения нагрузки на GPU.